```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("emails-Copy1.csv")
df.head() print(df.shape)
print(df.columns)
df.isnull().sum()
# Separate features (X) from the target state variable (y)
# We isolate the 3,000 text features to perform outlier processing
word_features = [col for col in df.columns if col not in ['Email No.', 'Prediction']]
X = df[word_features].copy()
y = df['Prediction']
plt.figure(figsize=(8, 5))
sns.boxplot(x='Prediction', y='the', data=df)
plt.title("Outlier Identification for Word Count ('the')")
plt.xlabel("Email State (0 = Not Spam, 1 = Spam)")
plt.ylabel("Word Frequency Count")
plt.show()
print("\n--- Identifying and Handling Outliers ---")

# Let's count how many total extreme outlier values exist across the entire dataset using the IQR method
total_outliers_count = 0
```

```
for col in word_features:
    Q1 = X[col].quantile(0.25)
    Q3 = X[col].quantile(0.75)
    IQR = Q3 - Q1
    upper_bound = Q3 + 1.5 * IQR

    # Count outliers in this specific column
    num_outliers = (X[col] > upper_bound).sum()
    total_outliers_count += num_outliers
    # Cap the extreme outliers at the 99th percentile to prevent data destruction
    upper_limit = X[col].quantile(0.99)
    X[col] = np.where(X[col] > upper_limit, upper_limit, X[col])

print(f"Total extreme data points identified as outliers: {total_outliers_count}")
print("Outliers successfully capped at the 99th percentile across all features.")
# Step 4: Check target values
print("\nPrediction value counts:")
print(df["Prediction"].value_counts())
# Step 6: Prepare X and y
# Remove Email No. because it is only an identifier
# Prediction is the target variable
X = df.drop(columns=["Email No.", "Prediction"])
y = df["Prediction"]
print("\nFeatures shape:", X.shape)
print("Target shape:", y.shape)
# Step 7: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
```

```
    stratify=y
)
print("\nTraining data:", X_train.shape)
print("Testing data:", X_test.shape)
# Step 8: K-Nearest Neighbors (KNN)
knn = KNeighborsClassifier(
    n_neighbors=5
)
# Train KNN
knn.fit(X_train, y_train)
# Prediction
y_pred_knn = knn.predict(X_test)
# Step 9: Evaluate KNN
knn_accuracy = accuracy_score(y_test, y_pred_knn)
knn_precision = precision_score(
    y_test,
    y_pred_knn,
    zero_division=0
)
knn_recall = recall_score(
    y_test,
    y_pred_knn,
    zero_division=0
)
knn_f1 = f1_score(
    y_test,
    y_pred_knn,
    zero_division=0
)
print("\n================================")
print("KNN PERFORMANCE")
```

```
print("================================")

print("Accuracy :", knn_accuracy)
print("Precision:", knn_precision)
print("Recall   :", knn_recall)
print("F1 Score :", knn_f1)
print("\nKNN Classification Report:")
print(
  classification_report(
    y_test,
    y_pred_knn,
    target_names=["Not Spam", "Spam"],
    zero_division=0
  )
)
# Step 11: KNN Confusion Matrix
cm_knn = confusion_matrix(y_test, y_pred_knn)
print("KNN Confusion Matrix:")
print(cm_knn)
plt.figure(figsize=(6, 4))
sns.heatmap(
  cm_knn,
  annot=True,
  fmt="d",
  xticklabels=["Not Spam", "Spam"],
  yticklabels=["Not Spam", "Spam"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("KNN Confusion Matrix")
plt.show()
```

```
# Step 12: Support Vector Machine (SVM)
svm = SVC(
    kernel="linear"
)
# Train SVM
svm.fit(X_train, y_train)
# Prediction
y_pred_svm = svm.predict(X_test)
# Step 13: Evaluate SVM
svm_accuracy = accuracy_score(y_test, y_pred_svm)
svm_precision = precision_score(
    y_test,
    y_pred_svm,
    zero_division=0
)
svm_recall = recall_score(
    y_test,
    y_pred_svm,
    zero_division=0
)
svm_f1 = f1_score(
    y_test,
    y_pred_svm,
    zero_division=0
)
print("\n================================")
print("SVM PERFORMANCE")
print("================================")
print("Accuracy :", svm_accuracy)
print("Precision:", svm_precision)
print("Recall   :", svm_recall)
```

```
print("F1 Score :", svm_f1)
# Step 14: SVM Classification Report
print("\nSVM Classification Report:")
print(
  classification_report(
    y_test,
    y_pred_svm,
    target_names=["Not Spam", "Spam"],
    zero_division=0
  )
)
# Step 15: SVM Confusion Matrix
cm_svm = confusion_matrix(y_test, y_pred_svm)
print("SVM Confusion Matrix:")
print(cm_svm)
plt.figure(figsize=(6, 4))
sns.heatmap(
  cm_svm,
  annot=True,
  fmt="d",
  xticklabels=["Not Spam", "Spam"],
  yticklabels=["Not Spam", "Spam"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("SVM Confusion Matrix")
plt.show()
# Step 16: Compare KNN and SVM
results = pd.DataFrame({
  "Model": ["KNN", "SVM"],
  "Accuracy": [knn_accuracy, svm_accuracy],
```

```
    "Precision": [knn_precision, svm_precision],
    "Recall": [knn_recall, svm_recall],
    "F1 Score": [knn_f1, svm_f1]
})
print("\n================================")
print("MODEL COMPARISON")
print("================================")
print(results)
# ============================================================
# Step 17: Performance Comparison Graph
# ============================================================
results.set_index("Model").plot(
    kind="bar",
    figsize=(8, 5)
)
plt.title("KNN vs SVM Performance Comparison")
plt.xlabel("Classification Algorithm")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend()
plt.tight_layout()
plt.show()
```

outliers: **Step 1: We Identified the Extreme Values**

Using the data distribution shown in your original boxplot (where some emails had a word frequency of over 200), we calculated the **99th percentile** for every word column.This means we found the threshold where 99% of the emails sit normally, identifying anything above that threshold as a severe outlier.

Step 2: We Capped Them (Winsorization) Instead of Deleting Them

Instead of deleting rows (which would ruin your dataset size) or leaving them alone (which would break the machine learning math), we **clamped** them.

The Logic: If the 99th percentile threshold for the word *"the"* was a frequency count of 40, any email that had the word *"the"* 50, 120, or 210 times was automatically set down to exactly 40.

Why this works: It preserves the "signal" that the word appears a lot in that email, but it removes the extreme mathematical weight that distorts your models.

Step 3: We Scaled the Data (StandardScaler)

After capping the extreme values, we applied StandardScaler right before training KNN and SVM.

- This shifted the mean of every column to 0 and scaled the variance to 1.
- This final normalization step completely leveled the playing field across all 3,000 features, ensuring no single high-frequency word could dominate the distance calculations.

Classifier	True Negatives (Correctly identified regular emails)	True Positives (Correctly caught spam)	False Positives (Legitimate emails sent to spam) ⚠️	False Negatives (Spam emails missed in inbox) 📥
KNN	648	254	**87**	**46**
SVM	**716**	**285**	**19**	**15**

Accuracy measures the proportion of **total correct predictions** (both normal and spam) out of all evaluation data points. $$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Precision focuses entirely on the **reliability of the positive predictions** (spam alerts).tp/tp+fp. High precision prevents **False Positives**—ensuring important, legitimate emails are not accidentally thrown into the junk folder.

Recall focuses entirely on the **model's ability to capture all actual positive states** (catching real spam).**Formula:** Recall= TP /TP+FN (High recall prevents **False Negatives)**

The F1-Score represents the **harmonic mean of Precision and Recall**, blending them into a single metric.**Formula:**F1-Score=2* (precision*recall/pre+recall).. How well does the model maintain a balance between catching all the spam (Recall) and not making false alarms (Precision).

Supervised Learning

The model learns from **labeled historical data** (data where the correct answers are already provided). Its goal is to find relationships between the input features and the output label to make predictions on new, unseen data.

Regression Example: Predicting house prices based on square footage, location, and the number of bedrooms.

Classification Example: Your email spam filter project. The algorithm checks historical emails marked as "Spam" or "Not Spam" to classify new incoming messages.

Unsupervised Learning

The model is given **unlabeled data** with no target answers. Its goal is to find hidden patterns, groupings, or structures within the data on its own.

Clustering Example: E-commerce customer segmentation. A company groups millions of shoppers into distinct personas (e.g., "bargain hunters" vs. "luxury spenders") based entirely on their browsing and spending patterns without any pre-defined labels.

Anomaly Detection Example: Credit card fraud systems detecting an unusual purchase location that does not match a user's typical routine behavior.

Reinforcement Learning

The model learns through a system of **trial and error, rewards, and punishments** by interacting with a dynamic environment to maximize a long-term goal.

- **Autonomous Driving Example:** Self-driving cars learning how to navigate traffic lanes, change speeds, and avoid collisions based on positive or negative reinforcement signals from simulation environments.

- **Gaming Example:** Google DeepMind's AlphaGo learning to defeat world champions by playing millions of games against itself and scoring points for winning moves.

KNN is a supervised machine learning algorithm that predicts the output of new data by looking at the K closest data points (neighbours).Real-time Example: Movie Recommendation,product,disese prediction,handwriting recognition,house price,music recom

Simple Definition: SVM is a supervised machine learning algorithm that classifies data by finding the best boundary (hyperplane) to separate different categories.Real-time Example: Email Spam Detection,cancer classi,face recogi,fraud detection,sentiment analysis

KNN	**SVM**
Uses nearest neighbours.	Uses the best separating boundary.
Distance-based algorithm.	Boundary-based algorithm.
Easy to understand.	Comparatively complex.
Prediction can be slower for large datasets.	Can work well with high-dimensional data.